Corrected Answers

Introduction to IaC and first deployment with Terraform

Q1 What is Infrastructure as Code? Name three advantages over a manual configuration.

IaC means defining and managing infrastructure with configuration files or code, instead of creating resources manually. An IaC tool interprets the description and performs the operations.

- **Automation:** fewer manual, repetitive operations.
- **Repeatability:** the same infrastructure can be reproduced from a configuration.
- **Versioning:** changes are tracked with Git.

Also valid: consistency between environments, traceability (review before applying), collaboration.

Q2 What is the difference between an imperative and a declarative approach?

- **Imperative:** you describe the actions to run and their order (create a network, then a VM, then attach it).
- **Declarative:** you describe the desired result and the tool determines the necessary operations. Terraform is mainly declarative.

Q3 What is the role of Terraform?

Terraform is HashiCorp's IaC tool. It defines, provisions and manages infrastructure resources from configuration files, and communicates with platforms through providers.

Q4 What is a Terraform provider? Give two examples.

A provider is a plugin that lets Terraform interact with a platform or service, translating its work into API calls.

Examples: **AWS** and **Microsoft Azure** (also Google Cloud, Docker, Kubernetes, VMware vSphere).

Q5 What is the difference between a resource and a data source?

- **Resource:** a component Terraform creates, modifies or deletes (e.g. `aws_instance`).
- **Data source:** reads information about an existing resource or object, without necessarily creating it.

Q6 What is the role of the terraform init command?

It initializes the working directory: it downloads the required providers and prepares the `.terraform` directory. It is the first command to run.

Q7 What is the difference between terraform plan and terraform apply?

- `plan` shows the changes Terraform intends to make by comparing the configuration with the state and the remote infrastructure. It changes nothing.
- `apply` carries out the changes needed to reach the desired state. It displays a plan and asks for confirmation, unless options say otherwise.

Q8 Does terraform validate guarantee that the resources can be created on AWS? Justify.

No. `validate` only checks the syntactic and structural consistency of the configuration. It does not contact AWS, so problems such as missing permissions, quotas, an invalid AMI ID or the wrong region only appear at plan or apply time.

Q9 What is the role of the terraform.tfstate file?

It records the resources managed by Terraform and maps the configuration to the real objects. Terraform uses it to work out what must be created, changed or deleted.

Q10 Why should the State file not be published in a public GitHub repository?

It describes the real infrastructure and can contain sensitive information (identifiers, IP addresses, sometimes secrets in clear text), which anyone could read in a public repository.

Good practice: add it to `.gitignore` and use a protected remote backend.

Q11 In the following block, identify the resource type, its logical name and the provider used.

```
resource "aws_instance" "web" {
  instance_type = "t3.micro"
  ami           = "ami-xxxxxxxx"
}
```

- **Resource type:** `aws_instance`
- **Logical name:** `web`
- **Provider:** `aws`, deduced from the prefix of the resource type.

Q12 Why is it important to run terraform plan before terraform apply?

> To review what will be created, modified or destroyed before anything touches real resources. It lets you catch mistakes (unwanted deletions, wrong environment) and provides traceability and safety.

Workflow at a glance

Command	Role
`terraform init`	Initialize the project and download providers
`terraform fmt`	Format the configuration
`terraform validate`	Check syntax and structural consistency
`terraform plan`	Preview the changes
`terraform apply`	Apply the changes
`terraform destroy`	Destroy the managed resources (check the target first)